# Week 7 逐页中文讲稿 · 卷积神经网络与计算机视觉应用

**课程**：52015CC3BV AI 应用开发 · 模块二 W5–8 · Week 7 **课件**：lectures/week-07/lecture-zh.pptx（32 页） **总时长**：讲授段 90 min（复习导入 10 + 新课讲授 80） + 小结 10 min；实验段 80 min 另立教案 **写法说明**：这是**照着讲的话术**，第一人称、可直接读；每页给建议时长；方括号内是**动作提示**（板书、演示、提问、停顿），不必念出来。 **数字纪律**：讲稿里每个数字都来自 outputs/w7_cnn.json（5 架构 × 5 种子 + 平移探针 + 混淆矩阵 + 累积探针 + 内存扫描）。**上课前请重跑 scripts/w7_cnn_experiment.py 确认一遍**；若 JSON 变了，讲稿里的数字要跟着改。

---

## 时间总表

| 环节 | 页码 | 时长 |
|---|---|---|
| 一 · 复习与导入 | p1–p4 | 10 min |
| 二 · 7.1 卷积、池化与感受野 | p5–p15 | 40 min |
| 三 · 7.2 训练、鲁棒性与错分 | p16–p24 | 40 min |
| 四 · 显存与绿色计算 | p25–p29 | 10 min（p26—p28 精讲，p25/p29 略讲） |
| 五 · 小测与小结 | p30–p32 | 10 min |

⚠ 教案里写”课件共 29 页”是**旧数**；实际课件已扩到 **32 页**（加了 3 页矢量示意图）。讲稿按实际的 32 页走。实验段（80 min）对应 p22–p26 的实操内容，另见实验指导书。

---

## 一 · 复习与导入（p1—p4，10 min）

### p1 · 封面

**时长 0.5 min**

早上好。今天是第七周，模块二的核心部分。

前面六周我们一直在做一件事：把数据变成向量，喂给全连接层。这周开始换一条路——**图像不再被铺平成向量。**

但我先说清楚，这周**不是**”卷积比全连接好”的课。这句话我自己都不信。为什么不信，等第 18 页我会用数据告诉你们。

今天这 180 分钟，课程的官方进程是四段：复习导入 10 分钟，新课讲授 80 分钟，随堂实验 80 分钟，小结作业 10 分钟。我负责前两段和最后一段，中间 80 分钟你们做实验 6，我巡场。

[停顿，看向学生]

**要点**：开场就把”这周不做什么”说清楚——本周主线是**参数效率与鲁棒性的代价**，不是”CNN 更准”。

---

### p2 · 目录

**时长 1 min**

先看今天的路线图，五个部分。

第一部分，为什么不能把图像铺平——这件事听起来老掉牙，但我保证你们看完会觉得跟自己想的不一样。

第二部分，形状的算术。卷积、步长、填充、池化、感受野。这部分是**算术**，不是玄学，今天你们要学会自己算。

第三部分，训练起来：曲线、增强，以及增强的代价。

第四部分，看它错在哪。这是我今天最想跟你们聊的一段。

第五部分，显存。**显存不够几乎从来不是硬件问题**，这句话我今天会用数据证明给你们看。

课程目录里还有一句：本周的主线不是”CNN 更准”。我把它念出来——**主线是：CNN 用 1/9 的参数拿到同样的准确率，而且更抗平移；但为”抗平移”付的钱，会直接从干净测试集的分数里扣。**

这句话有两半，前半是收获，后半是代价。今天你们要同时拿到这两半。

[用手势指屏幕上那句主线]

**要点**：把主线读出来并预告”代价”这一半，为 p20 埋线。

---

### p3 · 章节页：卷积——从全连接到权值共享

**时长 0.5 min**

第一个部分。

这里有三个词：局部连接、权值共享、参数可控。

我先给结论：卷积做的**不是**"更聪明的分类器"，它是对**参数的数量**动手。

很多同学学卷积，脑子里想的是"卷积能提取特征，提取的特征更高级"。这个理解不算错，但它会让你学得很辛苦。因为卷积真正在做的事，跟"高级"没关系——它是在**省参数**，省的方式是**赌一个假设**：这个假设叫局部性。

等一下我会告诉你们这个赌注在什么情况下会输。

[翻页，不在此页停留]

---

## p4 · 为什么不能直接把图像铺平

**时长 8 min**（本页是导入段的重头戏）

我们从最朴素的做法开始：把 8×8 的图铺平成 64 维向量，喂给全连接层。

先说结果，这个做法在 8×8 上勉强能跑，在真实图像上会**立刻崩掉**。三个理由，每一个我们这周都有实验数字能验。

**理由一，参数会爆炸。**

同一份 digits 数据，全连接网 17,226 个参数，两层卷积只要 1,898 个。差 9 倍。

有人可能会说，8×8 才 64 个输入维度，17,226 个参数有什么了不起？

关键在于**这个数字随图宽线性增长**。输入是 8×8 就 64 维；真实图像是 224×224，49,152 维。第一层权重就是 49,152 乘以隐藏层宽度。**你的显卡不是不够大，是你的参数量还没开始算就已经荒谬了。**

[板书] 参数量 ∝ 输入维度 × 隐藏维度。图像的输入维度是几万像素。

**理由二，铺平会丢掉空间结构。**

8×8 里相邻的两个像素，和隔着整行的两个像素——铺平之后，在全连接层眼里**完全一样**。它们只是向量的第 7 位和第 15 位。

但对一张图来说，这两个像素的关系是本质区别。卷积层保留了这个结构——**而且这正是它能共享权重的前提**。你想想，如果每个位置要单独学一遍参数，那就不叫共享了。

**理由三，平移不该被当成"新样本"。**

这条我用实测说话：同一手写数字，我把它**右移 1 列**，权重一个字都不动，全连接网准确率从 0.9785 掉到 0.5089。

掉了一半。

为什么会这样？因为全连接网**必须为每个位置单独学一遍**。它见过"这个笔画像这样出现在第 3 行"，但它没见过"同样的笔画像这样出现在第 4 行"。你把图挪一格，它就抓瞎了。

[停顿] 卷积做的事，是把"同一个模式出现在哪里"这件事，**写进了结构里**。

[停顿 2 秒，扫视全班反应]

**要点**：三条理由各自对应"参数爆炸 / 丢结构 / 不抗平移"。第 ③ 条的实测数字（0.9785 → 0.5089）要在 p19 再用一次，形成呼应。

---

## 二 · 7.1 卷积、池化与感受野（p5—p15，40 min）

### p5 · 卷积：一次滑动的点积

**时长 4 min**

现在我们把卷积拆开看。屏幕上这个是**真实的 digits 数据**，不是示意图。

卷积在做的事情，用一句话说完就是：**取一块，逐元素相乘，求和。**

[用激光笔或手指指屏幕] 中间这个 3×3 的核，是竖直边缘检测核：左边一列加 1，中间列是 0，右边一列减 1。

你看左边输入图的中间那几行：0、0、8、8、0、0 这样排。当这个核落到高亮窗口的位置上，**左列乘 1、中列乘 0、右列乘 1**——注意这里是"加右边减左边"，所以窗口里的 8、8、0 被加，0、0 被减，结果是 16。

但屏幕上写的是 9 和 8，对吧？这就是**逐元素相乘再求和**的结果，不是简单加减。每一个位置都是：窗口里 9 个数，分别乘核里对应的 9 个数，然后加起来。

核滑到下一个位置，再算一遍。6×6 的输入、3×3 的核，得到 4×4 的输出——**尺寸是这么来的：6 减 3 加 1 等于 4。**

右边输出的每一个数字，都能追溯到左边某一个 3×3 窗口。虚框是核下一步要落的位置。

[停顿] 所以如果我只给你一句总结：**卷积的全部算术，就是"取一块、逐元素相乘、求和"重复若干次。剩下的都是工程。**

但正因为它是这么朴素的一重复，才有了后面所有的效率性质。

**要点**：强调”逐元素相乘再求和”而非加减——学生最容易在这里理解错。尺寸 6→4 的来源点出来，为 p10 铺垫。

---

## p6 · 卷积就是一次滑动点积——先把这句话写成代码

**时长 4 min**

我知道你们接下来要用 nn.Conv2d，但我还是想花五分钟写一遍朴素版。

**不是**让你们以后都手写。手写是为了让”特征图”“步长”“填充”这三个词有**具体含义**。

[指代码] 上面这个 conv2d_naive，核心就三行：算输出尺寸 oh、ow；开一个输出张量；两重循环，滑窗、取块、乘、求和。

下面这段更有意思。我没有说”我这个实现是对的”，我写的是**断言**：

```
assert mine.shape == ref.shape == (8, 8)
assert torch.allclose(mine, ref, atol=1e-5)
```

意思是：我拿 PyTorch 官方的 nn.Conv2d，把权重塞成我那个核，然后**断言我的实现和官方逐项一致**。

这个习惯请你们带走。在这个课程里，我们不说”我觉得它对”，我们说”我断言它对，不一致就崩”。

[指补零那几行] 还有一个小细节：pad 的时候是先 pad 再算尺寸，所以公式里那个 2p 是这么来的。

好，跑通这一页你就知道了——**卷积没有魔法**。

**要点**：教学重点是”用断言对齐，不靠相信”——这与课程”证据链”要求一致，不要讲成”手写实现很重要”。

---

## p7 · 参数量：逐层对账

**时长 3 min**

现在我们把账算清楚。**参数量是代码数出来的，不是估算的。**

[指表格上半] 这是全连接网：第一层 64 到 128，8,320 个参数；第二层 128 到 64，8,256；最后一层 64 到 10，650。累计 **17,226**。

[指表格下半] 这是卷积网：第一层卷积 1 到 8、3×3，80 个；第二层卷积 8 到 16、3×3，1,168；线性头 650。累计 **1,898**。

我念一下这两个卷积层是怎么算的，好让你们知道表格不是我编的：

- 第一个卷积：3 乘 3 乘 1 个输入通道乘 8 个输出通道，加 8 个偏置，**80**。
- 第二个：3 乘 3 乘 8 乘 16，加 16，**1,168**。

[指最后一行] 最后一层 64 到 10。64 是哪来的？16 个通道乘 2 乘 2，池化之后是 2×2，flatten 就是 64。

下面这句话是这张表的灵魂，我念一遍：

**17,226 减到 1,898，少 9 倍。而两者的测试准确率是 0.979 和 0.981——差 0.002，不到一个标准差。**

停一下。**参数少了 9 倍，准确率基本一样。**

这才是卷积的贡献。它没有让你的模型变聪明，它是让你**用 1/9 的代价拿到同样的结果**。

[停顿 3 秒，让”9 倍”落地]

**要点**：本页是全周最重要的一张表。**参数量 9 倍差距 vs 准确率 0.002 差距**，这个反差要停下来。

---

## p8 · 权值共享买到了什么，又付出了什么

**时长 3 min**

上一页说卷积是”用假设换参数效率”，那我们把账正反两面都算清楚。

**买到的**：

第一，参数量**与图像宽度无关**。同一个 3×3 核滑过 8×8 和滑过 1024×1024，参数数量一模一样。这就是”参数量与图宽无关”的准确含义。

第二，“同一个模式在任何位置”这件事，被**写进了结构**，不是靠数据学出来的。实测：平移一列，卷积网 0.6481，全连接 0.4555——**高 19 个点**。

**付出的，三条**：

第一，**局部性是假设，不是真理**。如果你的有用模式是全局的，比如”整张图的亮度分布”，这个假设就是错的，**卷积网会比全连接更差**。

第二，**感受野是有限的**。这一点下一页专门讲。

第三，**池化会丢位置信息**，而有些任务恰恰要位置——分割、检测、关键点。这是后面几周的事。

一句话记法，请记这一句：

**卷积不是"更强的层"，是"一个关于数据结构的假设换来的参数效率"。假设错了，它比全连接更差。**

**要点**："假设错了比全连接更差"这句必须说出来——这是给实验 6 埋伏笔，也是学术诚实。

---

## p9 · 章节页：形状的算术

**时长 0.5 min**

第二部分，形状的算术。

我先说一句：**形状错误是深度学习里最常见的报错，也是最容易一次学会的。**

我教过很多届学生，90% 的 shape mismatch 报错，都出在这一页上的三条公式。学会了这一页，你们能自己定位。

下面每一条公式，我都在**真实张量上验算过**，不是我口头说一致。

[翻页]

---

## p10 · 输出尺寸公式，与真实张量逐项核对

**时长 4 min**

公式：**out = (size + 2p − k) / s + 1**，向下取整。

[指代码注释] 这里给了常用取值速查，输入都是 8×8：

- k=3, s=1, p=1 → **8**，尺寸不变，这是"same"卷积；
- k=3, s=1, p=0 → **6**，每层小 2；
- k=2, s=2, p=0 → **4**，池化，减半；
- k=3, s=2, p=1 → **4**，下采样；
- k=1, s=1, p=0 → **8**，1×1 卷积，只换通道不换尺寸。

这五个我背不下来没关系，**你们要会算**。

[指下半部分] 下面是真正重要的部分。我把本週的四个阶段写成列表，然后拿公式算一遍，再拿**模型真实前向**跑一遍，两个对照。

看这个输出：conv → 8，pool → 4，conv → 4，pool → 2。左边是公式算的，右边是前向跑出来的，**两列一样**。

强调一下：这个"一样"是**脚本里的断言**，不是我今天站在这里保证的。我要是写错了，脚本会崩。

[停顿] 所以下次你报 shape mismatch，**先算这一页，不要先改代码**。90% 的形状错误在这一页就能定位。

**要点**：强调"先算再改代码"。速查表让学生拍照，不要求背。

---

## p11 · 感受野：一张 8×8 的图，两层就看完了

**时长 3 min**

感受野是这一节最关键的概念，我用图讲。

屏幕上**三个格子都是同一张 8×8 的输入**，阴影部分表示"那一层一格的输出，能看见输入的哪一块"。

第一个，卷积 1，k=3。一格输出看见 3×3。

第二个，最大池化，k=2, s=2。**4×4**。注意这里的关键：**步长 2 把 jump 翻倍了**。池化不是白白缩小，它是让后面每一层"看得更远"。

第三个，卷积 2，k=3。**8×8——整张图，到此为止。**

[指第三个格子] 所以第二层卷积的时候，一个输出已经能看到整张输入了。

那我多叠几层卷积，能不能看到更多？**不能。** 数据只有 8×8，全图就 8×8。

下一页我们上公式。

**要点**：用"三个格子是同一张图"这个视觉抓手。jump 翻倍是理解感受野递推的钥匙。

---

## p12 · 感受野：每一层的输出，到底看了输入多少

**时长 4 min**

上公式。感受野的递推是：**rf = rf + (k − 1) × jump**，而 jump 每层乘以 stride。

[逐行指表格]

输入：感受野 1，jump 1。

卷积 1，k=3：1 + 2×1 = **3**，特征图还是 8×8，jump 还是 1。

池化，k=2, s=2：3 + 1×1 = **4**，特征图 4×4，**jump 翻倍成 2**。

卷积 2，k=3：4 + 2×2 = **8**，特征图 4×4，jump 2。

池化：8 + 1×2 = **10**，特征图 2×2，jump 4。

感受野序列：**3 → 4 → 8 → 10**。

[加粗语气] 现在看**加粗那一行**——卷积 2，感受野 8。

**8 正好等于整张图。**

换句话说：在本周这个 8×8 的数据上，**第二层卷积已经能看到全图了**。这个网络不可能再靠"多叠卷积"获得更多的平移不变性。

我要强调一句：**这不是模型不好，是数据太小。**

你们以后看论文里说"我们的网络有 50 层，感受野很大"，要知道那是在 224×224 的图上算的。这里数据只有 8×8，你叠到第 2 层就已经封顶了。

[停顿] 感受野这个概念真正的作用是：**它让你能算出一个任务需要几层。** 到这里你们应该能自己算了。

**要点**：三处强调——"加粗那一行"、"数据太小不是模型不好"、"感受野用来算需要几层"。

---

## p13 · 池化：四个数变成一个

**时长 3 min**

池化。同一个 4×4 输入，同一个 2×2 不重叠窗口，**只有规则不同**。

上面是最大池化，取四个里最大的，2、5、3、7 这块，出 7。

下面是平均池化，取四个的平均，出 2.5。

我特意把它们并排放，就是想让你们看到：**输入完全一样，窗口完全一样，输出差这么多。**

然后是本周的实测：除了池化函数，**两个架构逐字节相同**——同样的通道数、同样的核、同样的学习率、同样的轮数。最大池化比平均池化**高 4.1 个点。**

4.1 个点是什么概念？**是 5 个标准差的 5 倍。这不是噪声。**

为什么？手写数字的判别信息，**正好落在笔画最集中的地方。**取最大，保住了它；取平均，把它摊薄了。

一句话：**选池化函数，是在选择你愿意保留什么。**

**要点**：“逐字节相同”是这页的实验设计亮点——排除了其他所有变量。

---

## p14 · 池化在做什么——三个作用，一个代价

**时长 3 min**

池化到底在干什么？教科书常说”为了降维”，这只是三个作用里**最常被当成全部**的那一个。

**作用一，缩小特征图。** 8×8 到 4×4 到 2×2，后续层的算力和显存按面积下降。

**作用二，扩大感受野。** 这条更重要，也最容易被忽略。stride 2 的池化让 jump 翻倍——**没有它，你叠再多 3×3 也只有 3、5、7……**。看上一页，加上池化之后第二层卷积就覆盖了整张图。

**作用三，引入小范围容忍。**“这一片里有强响应”，比”正好第 3 行第 5 列有强响应”更稳。抗的是小幅抖动。

**但代价在最后一格：丢位置。**

需要”在哪里”的任务——分割、检测、关键点——必须把位置信息补回来。那是后面几周的事。

[停顿] 所以池化不是免费的。**它拿位置换容忍度。**

**要点**：明确”代价在最后一格”，为后续周次留接口。

---

## p15 · 池化方式的选择，值 4.1 个点

**时长 2 min**

这一页是上一页的表格版。数据我念一遍：

- 最大池化：训练 0.999，验证 0.970，**测试 0.981。**

- 平均池化：训练 0.950，验证 0.929，**测试 0.940。**

参数量都是 1,898，前面说过，**除池化方式外两个架构逐字节相同。**

差 0.041，是标准差的 5 倍。

我想请你们注意一件事：**平均池化的训练准确率只有 0.950，验证和测试都同步低。** 这不是过拟合，也不是调参没调好——**是模型压根没学进去。**

这和下一页要讲的"增强让训练准确率降下来"是完全不同的机制。等一下你会看到，增强降的是过拟合，池化降的是**学习能力。**

[停顿] 记住这句话：**选池化函数是选择，不是默认。**

**要点**：训练 acc 同步低 ⇒ 不是过拟合，是没学会。这个区分在本页点出来。

---

## 三 · 7.2 训练起来：曲线、增强与代价（p16–p24，40 min）

### p16 · 章节页：训练起来——曲线、增强与代价

**时长 1 min**

第三部分。

先交代协议，因为不交代协议，后面的数字没法读：

**5 个架构，每个 5 个随机种子，42 到 46。** 同一套数据、同样的切分、同样的优化器、同样的轮数。

所有差异都写成"均值 ± 标准差"。

然后是我们的纪律：**差异不到一个标准差，就写"未观察到显著差异"。**

我知道这句话不好听。你跑了实验，看到 0.981 比 0.979 高，很想写"卷积更好"。但 0.002 减去一个标准差 0.008——**这个差，我们没有观察到。**

我今天会当着你们的面，把这句话用在最让人难受的地方。

[停顿，让"最让人难受的地方"这句话悬着]

---

## p17 · 验证准确率：三条曲线，同一个训练协议

**时长 3 min**

这是 seed 42 单次运行的验证准确率曲线，每 6 轮取一点。

三个要点：

**第一，三条曲线在 20 轮后全部进入平台，之后的翻转都是噪声。** 记住这句——**“曲线后面在抖”不代表”它在变好”。**

**第二，卷积网起步更慢。** 第 1 轮，卷积 0.31，全连接 0.70。**卷积比全连接慢得多。** 但 40 轮之后反超。

这条我要多说一句，因为它影响你们的直觉：卷积不是”收敛更快”，它是”起步更慢但上限差不多”。如果你只跑 20 轮就下结论，你会得到完全相反的结论。

**第三，增强那条曲线更低但更稳。** 这不是训练失败——**是它在解一个更难的问题。** 下一页给数据。

**要点**：“起步更慢但上限差不多”——这是反直觉点，也是防止早停结论错误的关键。

---

## p18 · 五个架构的总账

**时长 4 min**

这是本周的总表。我一行一行念，你们听出三件事。

**第一件，卷积 vs 全连接。**

全连接 17,226 参数，测试 0.979 ± 0.008。卷积 1,898 参数，测试 0.981 ± 0.008。

差 0.002。**标准差是 0.008。**

所以正确的写法是：**未观察到显著差异。** 不是”卷积更好”。B 选项在今天的随堂测里会出现。

**第二件，变宽。**

CNN 宽 16/32，21,312 参数——**已经是基准的 11 倍，也比全连接网还大。**测试 0.987，确实最好。

但你看这个交换比：多 11 倍参数，换 0.6 个点。**这不叫划算，这叫用参数换分数。** 而且它把卷积”参数少”这个唯一的优势彻底抹掉了。

**第三件，增强。**

0.981 掉到 0.950。**单独看是变差的。**

我知道你们在想"那为什么要用"。下一页给答案——但我要你们先接受一个事实：**单看这张表，增强是纯亏的。**

[停顿 2 秒]

**要点**：三件事的顺序不能乱。**"未观察到显著差异"这句要在这里第一次完整说出。**

---

## p19 · 把测试图平移一下，差距就出来了

**时长 4 min**

现在我们来看被藏起来的那一面。

做法很简单：**测试时把整张图平移 dx 列**，用 0 填充边缘，**权重一个字都不动**。每个格是 5 个种子的均值。

看 dx = −1 这一行：全连接 0.4555，卷积 0.6481，增强 0.9415。

再看 dx = 0：全连接 0.9785，卷积 0.9808，增强 0.9504。

[指 dx=0 那一列] 注意这里——**增强在 dx=0 那一列是三条里最低的。**

这就是完整的画面了：

- 卷积比全连接**更抗平移**，这是权值共享买来的。
- 增强把曲线**压平**了——它没有提高上限，它把整条曲线托住了一条。

[停顿] 请注意这个区别：卷积是**把曲线整体抬起来**，增强是**把曲线压平**。

一个是能力，一个是保险。

**要点**：两个比喻要分清——"抬起来"（卷积）vs"压平"（增强）。

---

## p20 · 增强的账本：买 +29 个点，付 −3 个点

**时长 5 min**（本周最值得记住的一页）

上一页我们看到了两条曲线，现在我们把账算清楚。这一页我讲得慢一点。

买到了什么：

dx = −1：从 0.6481 到 0.9415，**+29.3 个点**。

dx = −2：从 0.2348 到 0.5170，**+28.2 个点**。

而且训练曲线不再冲上 1.000——第 115 轮只有 0.9292。**过拟合被压住了。**

付出了什么：

dx = 0：从 0.9808 到 0.9504，**−3.0 个点**。

训练准确率也从 0.9912 掉到 0.9292。**注意：训练和测试同步掉。** 这意味着**代价不是噪声，是系统性的。**

真正贵的地方在这儿：多花的时间可以忽略——120 轮里每轮多做两次随机平移，多几毫秒。**真正贵的是你得重新调正则和轮数**，因为模型行为变了。

[停顿，然后说出结论]

所以"加增强对不对"**没有统一答案**。

它问的是一个**产品问题**，不是模型问题：

- 如果你的用户是拿手机拍的——会平移——那就买。
- 如果你的输入已经被裁齐、对齐了——不会平移——**那你就是在白买**，还白掉 3 个点。

这是今天第一节课我希望你们带走的东西：**技术选择要回到使用场景。**

[停顿 3 秒]

**要点**：本页是全周的高潮。三处停顿 + 最后把技术问题落到产品问题。

---

## p21 · 章节页：看它错在哪

**时长 1 min**

第四部分，看它错在哪。

我先说一句你们可能不爱听的话：**"98%" 不是一个结果，是一个不完整的句子。**

准确率 0.98，这句话缺了什么？缺了**它在哪些类之间犹豫**。

混淆矩阵就是把"98%"翻译成"它在哪两类之间犹豫"的那张表。

屏幕上三个数字：1350 个测试预测（5 个种子加起来），**26 个非对角错误**，最差的类是数字 8，召回 0.9538。

26 个错误里，**12 个跟数字 8 有关**。我们去看。

[停顿]

---

## p22 · 混淆矩阵

**时长 4 min**

混淆矩阵的读法只有一句：**看非对角。**

行是真实标签，列是模型判断，对角是正确，非对角是错误。

合计 1350，正确 1324，0.9807。

我不念整张表，我挑三处：

**第一，数字 0 是满分。** 135 比 135。

**第二，数字 8 最差。** 对角 124，它那一行一共 130，所以召回 0.9538。

**第三，看第 8 行。** 有 3 个被判成 1，1 个被判成 2，1 个被判成 5，1 个被判成 7。

所以**最差的类对是 8 → 1，出现 3 次**。然后 1 → 8 和 1 → 9 各 2 次，6 → 8 是 2 次。

[加粗语气] **全部 26 个错误里，有 12 个与数字 8 有关，占 46.2%。**

现在你们看到了一件很反直觉的事：**0.981 这个数字，一个字都没提 8 有问题。**

准确率是"平均分"，它把难的和易的一起平均掉了。

**要点**："26 个错误里 12 个与 8 有关"是本页唯一要记住的数字。

---

## p23 · 把错分的那个数字本身画出来

**时长 5 min**

现在我们不只看数字，**把错分的那个数字本身画出来。**

灰度用字符表示：空格是 0，@ 是最亮。**这些是真实测试样本的像素，不是示意图。**

[指第一个] 第一个：**真 8，被判成 7，置信 0.9829。**

你自己看这个像素图——**下半部分被写断了，上部只剩一撇。人眼看也像 7。**

而且模型给 7 的概率是 0.9829，**给真类 8 的概率只有 0.0144。**它不是犹豫，它是**自信地错了。**

[指第二个] 第二个：**真 6，被判成 8，置信 0.8852。**

这个 6 的**下半环是闭合的。**而 6 的下半环不该闭合——闭合了更像 8。

混淆矩阵里 6 → 8 出现 2 次，**这就是那 2 次长什么样。**

顺带看一眼这个 6 的真类概率：**0.0271。**模型给 8 的置信是 0.8852，**给真类 6 的只有 0.0271**——同样是不犹豫，同样是自信地错。

[停顿，语气转变]

所以这才是"错分分析"。

不是"模型还不够好"——**那是没办法修的。**

而是"**它在一个下半环闭合的 6 上会把 8 叫出来**"——**这是可以修的。** 补这类样本，或者调损失权重。

一句话：**错分分析的价值，是把"不好"变成"具体在哪不好"。**

[停顿 3 秒]

**要点**：本页是全周方法论的核心。**"自信地错了"**（0.9829 vs 0.0144）要单独强调。

---

## p24 · 错分分析的交付标准

**时长 3 min**

既然错分分析这么重要，我们把它定成标准。**五条，缺一条就是"只报了一个总数"。**

这正是你们实验报告最常见的失分点，我提前说。

**第一，打印混淆矩阵，不是只打印准确率。** 0.981 底下藏着 26 个错误，其中 12 个跟 8 有关——准确率一个字都没提。

**第二，点名最差的两个类，给出次数。** "8 被当成 1，3 次"，比"模型在某些类上表现较差"有用一百倍。

**第三，至少 3 个具体错分样本，附上模型概率。** 我要的是"真 8 判 7，置信 0.9829，真类概率 0.0144"这种句子。**概率能区分"犹豫"和"错了还自信"。**

**第四，每条错误配一个可检验的原因假设。**“这个 6 的下半环闭合”是假设，而且**可以检验**：找一批下半环闭合的 6，看是不是都被判成 8。

**第五，写出你打算试什么，以及怎么判断有没有用。** 同一个切分、同一个指标、**一次只改一处**——这是全课程不变的那条规则。

[停顿] 第五条最重要。前面四条是分析，第五条才是实验。

**要点**：五条是硬标准。第 ⑤ 条”一次只改一处”是全课程规则，单独强调。

---

## 四 · 显存与绿色计算 （p25—p29， 10 min)

### p25 · 章节页：显存与绿色计算

**时长 0.5 min**

最后一部分，显存。

我先说结论，因为它能帮你省下真金白银：

**显存不够，几乎从来不是硬件问题，是 batch size 的问题。**

下面四步，从上往下试：先降 batch，最便宜的一刀；再梯度累积，免费买回等效 batch；然后才是加显存；最后才是算力是有限资源这句话。

[翻页]

---

### p26 · batch size 与峰值内存：实测的线性关系

**时长 2 min**

这页是实测，不是估算。做法说明一下：输入用最近邻上采样到 32×32，**激活体量乘 16，跟真实视觉模型同量级**；单次前向加反向；**独立子进程测峰值 RSS。**

为什么要独立子进程？因为在同一进程里测，内存会被前面的分配污染，测不准。

[指数据]

基线——只导入 torch、不训练——**306.5 MB。**

batch 32 到 1024，增量从 83.6 涨到 275.5 MB，**3.3 倍，跟 batch 同步增长。**batch = 1024 时峰值 582 MB。

[停顿]

所以结论是：**峰值内存随 batch size 近似线性增长。**

而**下降 batch 是唯一一个"不改变数学、只改变内存"的旋钮。** 别的旋钮都会动你的结果，它不会。

**要点**：“唯一不改变数学的旋钮”是本页要记住的那句。独立子进程的做法点一句（体现严谨）。

---

## p27 · 梯度累积：等效 batch 可以被买回来，但要付对价钱

**时长 3 min**

降了 batch，训练效果会不会变？会——**梯度更吵了。**

梯度累积就是把这个效果买回来：用 4 个 micro-batch 拼出一个等效的大 batch。**数学上是等价的，内存上只占一个 micro-batch。**

但屏幕这张表说明了一件事：**“等价”这件事，有讲究。**

第一行是真 batch 128，基准。

**第二行，权重按"微批大小 ÷ 组大小"来写——5 个种子逐项完全一致。** 这才是正确的写法。

第三行，权重写成 1/累积步数——**只有 3/5 一致。**

第四行，忘了缩放——2/5。

[指第三行] 为什么第三行不完全一致？因为**每个 epoch 的最后一组往往不一样大。**n_train 是 1258，除不尽 64 乘以累积步数。简单平均会有偏。

[指第四行] 至于第四行为什么"看起来还行"？因为 **Adam 会按梯度的二阶矩做归一化，整体放大 4 倍几乎被它吃掉了。**

但我要加一句诚实的话：**换成 SGD 就完全不是这样了。** Adam 的这个特性是偶然的，不要依赖它。

**要点**：三行表 + 两个机制解释（最后一组不等大、Adam 的归一化）。“别依赖 Adam 的偶然特性”要说。

---

## p28 · OOM 的处置顺序

**时长 2 min**

把 p26 和 p27 串起来，就是这张表。**从上往下试，越往下代价越大。**

**第一步，免费**：batch size 减半。不改变任何数学，只改变内存。副作用是梯度更吵，用 2 倍步数或更高学习率补偿。**先做这个，别先怀疑硬件。**

**第二步，免费**：梯度累积，把等效 batch 买回来。实测 4×32 用对权重时与真 batch 128 **逐项一致**。代价是时间，不是精度。

**第三步，便宜**：混合精度 AMP。torch.amp.autocast 加 GradScaler，激活占一半。注意——**它改变数值**，所以要重新确认指标没漂。**不是免费，是便宜。**

**第四步，有代价**：梯度检查点或者更小的输入，通常慢 20–30%。缩小输入分辨率**会改变任务本身**——改之前想清楚你在测什么。

**第五步，最后**：降模型规模，或者承认这个任务本机做不了。

[加粗语气] **加显存是最后一条，不是第一条。**

本周实测：这个 8×8 模型峰值才 390 MB，batch 1024 也只要 582 MB——**它永远不会 OOM。**

所以第一件事是：**先算清楚你到底需要多少。**

**要点**：第 ⑤ 条"加显存是最后一条"是本页要带走的一句。AMP"不是免费，是便宜"要明确说。

---

## p29 · 绿色计算：算力是有限资源

**时长 1.5 min**

最后一页，我把这一周收在一个念头上。

[第一栏] **参数量就是电费。** 17,226 个参数拿 0.979，1,898 个参数拿 0.981。多出来的 15,328 个参数，在训练和推理里都要算、要传、要供电。

"用更少的成本达成目标"不是清高，是**先问一句：这个规模是必要的吗？**

[第二栏] **先量，再买。** 本周实测这个模型峰值 390 MB，batch 1024 也只要 582 MB。**在申请算力之前，先把峰值量出来。** 大多数"我需要更大的机子"，其实是"我把输入搞太大了"。

[第三栏] **无效的优化要如实写。** 本周的增强就是例子：它让干净精度掉了 3 个点。**要是只报平移实验那 29 个点，就是在选择性汇报。**

顺带说，这一条也是你们 AI 协同任务的要求：**AI 给的显存优化建议，实测之后，有效和无效都要标注。** 这就是本周作业的一部分。

[停顿]

一句话收尾：**少 9 倍的参数，一样的准确率，差 19 个点的抗平移——但抗平移是要付钱的，付的是干净精度上那 3 个点。**

这周你们要学的不是"卷积很厉害"，是**"每一个结构选择都有代价，而代价要能被算出来"。**

[停顿 2 秒]

**要点**："无效的优化要如实写"直接对接实验 6 的 AI 协同交付要求。收尾句把全周串成一句。

---

## 五 · 小测与小结 （p30—p32， 10 min）

### p30 · 随堂小测

**时长 4 min**

四道题，**不看笔记**，五分钟。

答不出的题，就是你本周末要补的地方。

[念题，可以只念题不念选项，让他们自己做]

**题 1**：8×8 输入，先 3×3 卷积 s=1 p=1，再 2×2 最大池化 s=2。输出特征图多大？换成 s=2 p=0 的卷积呢？

**题 2**：卷积网 1,898 参数拿 0.981，全连接 17,226 拿 0.979。写"卷积更好"对不对？

**题 3**：加了 ±1px 增强后，干净测试掉到 0.9504，平移 −1 时从 0.6481 升到 0.9415。这个改动有没有用？

**题 4**：要跑 batch 128 但显存只放得下 32。用 4×32 梯度累积，为什么权重不能简单写 1/4？

[计时 5 分钟，然后逐题讲评]

**题 1 答案 B。** 3×3 s=1 p=1：8×8 不变。池化 s=2：4×4。C 说"输出还是 4×4"——对，池化后就是 4×4，但第二问换的是**卷积**参数，s=2 p=0 的话是 4×4，所以 B 和 C 的区别在于第一问。B 完整。

**题 2 答案 B。** 差 0.002 小于一个标准差 0.008，**未观察到显著差异**，但参数量少 9 倍。A 是不懂标准差，C 是答非所问。

**题 3 答案 C。** 这是个产品问题——取决于真实输入里有没有平移。A 和 B 都只说了一半。

**题 4 答案 A。** 每个 epoch 最后一组的微批大小不等，简单平均会有偏。**注意第 4 题的 C 很诱人**——“完全等价，1/4 就行”——但实测只有 3/5 一致。差别就出在那个尾巴上。

**要点**：讲评时第 3 题和第 4 题要回扣到 p20 和 p27，讲”答案是 C，因为这是产品问题”。

---

## p31 · 本周小结

**时长 4 min**

六条，我念一遍，你们对照着记。

**一**，卷积不是”更强的层”，是”用局部性假设换参数效率”。同一数据上参数量差 9 倍，准确率差 0.002。

**二**，形状的算术必须先算再做。输出尺寸四条公式与真实张量逐项一致；感受野第二层就覆盖整图。

**三**，池化函数是**选择**，不是默认。最大池化与平均池化差 4.1 个点，是标准差的 5 倍。

**四**，增强买鲁棒性、付干净精度：+29 个点换 −3 个点。**用不用取决于输入会不会被平移。**

**五**，“98%”不是结果。混淆矩阵、最差类对、具体样本加概率、可检验的假设——**四件都齐才叫错分分析。**

**六**，显存不够先降 batch，再用梯度累积买回等效 batch。权重用”微批占组大小的比例”时，实测与真 batch 逐项一致。

[指课后待办]

三件事：

**实验 6，必交**：CNN 图像分类加数据增强；记录训练曲线、混淆矩阵、**至少 3 个错分案例，附概率和假设。**

**项目里程碑 M7**：推进准则是 python src/evaluate.py 能打印指标；**模型来源已记录**——用了谁家的预训练权重、什么许可；误差分析点名最差类对。

**课后预判**，写 5 行：如果我的图像变成 256×256，本周的架构会出现哪些新问题？提示：感受野、参数、显存、池化次数各想一遍。

[停顿]

最后一题我给个提示：256×256 是 8×8 的 **32 倍**面积，1024 倍像素。感受野那条你会怎么算，等下节课我检查。

**要点**：第 5 条说"四件都齐"——注意课件正文写的是四条（矩阵/类对/样本+概率/假设），第 24 页展开成五条（含"下一步"）。讲稿这里按课件正文说"四件"，避免与 p24 的五条冲突——如需统一可在这里补一句"加上第 24 页的下一步，共五条"。

---

### p32 · 结语与下周预告

**时长 2 min**

最后。

[念屏幕上的话] **"准确率高不高"是个不完整的问题。完整的问题是：它在哪两类之间犹豫，错的那几个长什么样。**

这句话你们如果只记一句，记这句。

今天所有数字都来自 scripts/w7_cnn_experiment.py → outputs/w7_cnn.json，5 个架构乘 5 个种子，加上平移探针、混淆矩阵、累积探针和内存扫描。**没有一份是我在黑板上算的。**

下周 Week 8，序列模型与文本数据处理：文本怎么变成张量、**填充与掩码为什么会吃掉你的精度**、LSTM 门控在解决什么。本周实验 7：分词、词表、嵌入、LSTM、分类。

谢谢大家。

[停顿，翻页]

---

## 附 · 教师使用提示

### 三个必须临场检查的点

1. **数字**：上课前跑一遍 python scripts/w7_cnn_experiment.py，确认 outputs/w7_cnn.json 与讲稿一致。本课程出现过"W07 三个数字凭记忆写、错误传播到 7 份材料"的事故（见 outputs/修订记录-W07-2026-10-09.md），**本讲稿所有数字都从 JSON 取，但 JSON 若重跑后变了，讲稿必须同步改。**
2. **课件与讲稿页码**：实际课件 32 页，教案里"29 页"是旧数（加了 3 页矢量图后未同步）。本讲稿按实际 32 页写。**若再改课件页序，讲稿要重排。**

3. **p31 的"四条"与 p24 的"五条"**：课件 p31 正文写"四件都齐"，p24 展开为五条交付标准。 讲稿已按正文说"四件"，讲到时**可补一句"加上第 24 页的第五条，共五条"**，避免学生觉得矛盾。

## 三处课堂互动点（举手/投票）

| 位置 | 提问 | 目的 |
|---|---|---|
| p4 讲完理由③后 | "全连接网右移一列掉到 0.5089——你觉得它掉一半正常吗？" | 引出"每个位置单独学" |
| p15 之后 | "平均池化训练 acc 也只有 0.950，这是过拟合还是没学会？" | 区分两种机制 |
| p18 之后 | "卷积 0.981 vs 全连接 0.979，差 0.002，标准差 0.008——这句话该怎么写？" | 强化"未观察到显著差异" |

## 时间风险点

| 页 | 计划 | 风险 | 压缩方案 |
|---|---|---|---|
| p7 参数量对账 | 3 min | 高（学生会算） | 公式已在 p6 铺垫，可只念结果 |
| p12 感受野 | 4 min | 高（递推易算错） | 不可压，这是本周难点 |
| p20 增强账本 | 5 min | 高 | **不可压，这是本周高潮** |
| p23 错分样本 | 5 min | 中 | 可只讲第一个样本（真 8 判 7） |
| p27 梯度累积 | 3 min | 高（四行表） | 可只讲第 2 行 + 最后一组不等大 |
| p29 绿色计算 | 1.5 min | 低 | 思政页，念结论即可 |

**总时长弹性**：讲授段 90 min 有 ±5 min 空间。若超时，优先压 p26（显存实测数据可只念结论）与 p29（思政可只念第一栏）。

## 与实验段（80 min）的衔接

p22–p26 的内容在实验段会**再用到**（混淆矩阵、错分分析、显存）。 讲授课段时点到为止，**留一个悬念**："这些表你们等下自己跑出来，我只给结论不给过程。"

## 数据溯源速查

| 讲稿页 | 数据来源 |
|---|---|
| p4 / p7 / p8 | outputs/w7_cnn.json → architecture |
| p15 / p18 | outputs/w7_cnn.json → arms |
| p19 / p20 | outputs/w7_cnn.json → translation_robustness |

| 讲稿页 | 数据来源 |
| --- | --- |
| p22 | outputs/w7_cnn.json → confusion_cnn_5seeds |
| p23 | outputs/w7_cnn.json → predictions_seed42 |
| p26 | outputs/w7_cnn.json → memory_vs_batch |
| p27 | outputs/w7_cnn.json → grad_accumulation |

*2026-10-26 · 讲稿逐页对齐 lecture-zh.pptx 实际 32 页；数字全部取自 outputs/w7_cnn.json，并与 lesson-plans/week-07/lecture-zh.html（教案 LP-W7-A）交叉核对；时长分配对齐教案「教学进程安排」四段。*